Laboratory assignment 1: REST API.

1. Install Node.js and npm from https://nodejs.org/en/download. Install Netlify globally with `npm install -g netlify-cli`.

2. - Choose one of the topics for personal or group assignment.
   - Generate repository based on a github classroom link provided to you. You will see a new repository generated - copy it locally with `git clone repo-link.git`.
   - Examine files in the repository. Take a look at default endpoint "hello"in `functions/hello.js`.
   - After running `netlify dev` and going to http://localhost:8888/.netlify/functions/hello you can see it being executed.

3. Your task is now to build a simple REST API with the following endpoints:

   - `GET /items` – List all items
   - `POST /items` – Add a new item
   - `GET /items/{id}` – Get details of a specific item
   - `PUT /items/{id}` – Update item info
   - `DELETE /items/{id}` – Remove an item

   Here instead of items you might use a more descriptive word specific for your task (for example, for budget planner app — `budgets`).

   You can add query parameters to the endpoints (pagination, sorting, filtering, etc.)

4. Create a simple frontend (html+css+js /React /Vue.js) that interacts with the REST API endpoints.

   Example of fetching data from API endpoint:

```
async function getData() {
  const url = "https://your-app-domain//items";
  try {
    const response = await fetch(url);
    if (!response.ok) {
      throw new Error(`Response status: ${response.status}`);
    }

    const json = await response.json();
```

```
    update(json); // update your front-end with data from
       back-end here
  } catch (error) {
    console.error(error.message);
  }
}
```

Using different request method (for example POST)

```
const response = await fetch("https://your-app-domain//items", {
  method: "POST",
  // ...
});
```

5. Connect your application to the database. You might use mongod.db for these purposes (prefered for this assignment).

   - Login to https://cloud.mongodb.com/ and create a cluster.
   - Generate a connection string for that cluster. Store it securely in `.env` file in your project. Do not forget to add `.env` to `.gitignore`.

   Example of how to connect to mongodb from your application, using connection string (you can name this file `db.js`):

```
const { MongoClient } = require("mongodb");

const client = new MongoClient(process.env.MONGO_URI);
let collectionName = "interview-questions";
let db;

async function connectDB() {
  if (!db) {
    await client.connect();
    db = client.db(process.env.DB_NAME);
  }
  return db.collection(collectionName);
}

module.exports = connectDB;
```

Example of how to set connection string in `.env` file:

```
MONGO_URI=mongodb+srv://your-user-name:mongo-passowrd@cluster-name/?re
```

Example of simple GET endpoint, that fetches data from mongodb and returns it to client:

```
const connectDB = require("./db");

exports.handler = async (event) => {
  try {
    const collection = await connectDB();
    const { topic, sortField, sortOrder = "asc", skip, take } =
       event.queryStringParameters;

    const query = topic ? { topic: new RegExp(topic, "i") } :
       {};
    const sort = sortField ? { [sortField]: sortOrder ===
       "desc" ? -1 : 1 } : {};

    const skipValue = Number.isInteger(parseInt(skip)) ?
       parseInt(skip) : 0;
    const takeValue = Number.isInteger(parseInt(take)) ?
       parseInt(take) : 20;

    const questions = await collection
      .find(query)
      .sort(sort)
      .skip(skipValue)
      .limit(takeValue)
      .toArray();

    return {
      statusCode: 200,
      headers: {
        "Access-Control-Allow-Origin": "*", // Allow all
           origins (change to specific domain later)
        "Access-Control-Allow-Methods": "GET, POST, PUT,
           DELETE, OPTIONS",
        "Access-Control-Allow-Headers": "Content-Type" // This
           part allows your to omit CORS checks. There will be
           tasks related to CORS later
```

```
      },
      body: JSON.stringify(questions) };
  } catch (error) {
    return {
      statusCode: 500,
      body: JSON.stringify({ error: error.message }) };
  }
};
```

---

6. Think of more elaborate error handling. Add different types of interconnected data (child-parent items, etc.).
7. Push code to reposiroty (you can use separate branch for this). Deploy your application to netlify.
    - Login to https://app.netlify.com/ and choose "Add new site"option
    - Link your github project. Specify environment variables (connection string same as in .env file). Specify "Build command"as `npm install`, "Publish directory"as `functions`, "Functions directory"also `functions`.
    - Deploy application and follow the link, when it is ready. Check, if everything works correctly. Do not forget to add "/.netlify/functions/"to address line if you have not specified otherwise.
8. Deploy frontend part of your application. In case of html+css+js stack you can use github pages.
9. Document your application in readme.md file. List contributors, how to run application locally, main idea of your app, etc.
10. Submit link to PR or commit and a link to your deploy as a result of your assignment work.

Laboratory assignment 2: GraphQL.

GraphQL is an open-source data query and manipulation language for APIs, and a runtime for fulfilling queries with existing data. GraphQL was developed internally by Facebook in 2012 before being publicly released in 2015.

Simply put, GraphQL is a query language for APIs and a runtime for executing those queries with existing data. GraphQL provides a complete and understandable description of the data in your API, gives clients the power to ask for exactly what they need and nothing more, makes it easier to evolve APIs over time, and enables powerful developer tools.

Implementing GraphQL endpoint to your backend application:

1. Install all the neccessary packages:
   `npm install serverless-http apollo-server-lambda graphql`
2. Implement schema for query and mutation. Example of `graphql.js` file:

```
const { ApolloServer, gql } = require("apollo-server-express");
const express = require("express");
const serverless = require("serverless-http");
const connectDB = require("./db");
const { ObjectId } = require("mongodb");

const typeDefs = gql‘
  type Item {
    _id: ID!
    fieldOne: String!
    fieldTwo: Int!
  }

  type Query {
    items(skip: Int, take: Int, sortField: String, sortOrder:
       String): [Item]
    item(id: ID!): Item
  }

  type Mutation {
    createItem(fieldOne: String!, fieldTwo: Int!): Item
    updateItem(id: ID!, fieldOne: String!, fieldTwo: Int!): Item
    deleteItem(id: ID!): Boolean
  }
‘;
```

Change field names to fields you have implemented for your custom data types.

3. Implement query and mutation resolvers in the same file. Example:

```
const resolvers = {
    Query: {
        items: async (_, {filter, skip = 0, take = 10,
           sortField = "fieldOne", sortOrder = "asc" }) => {
            const collection = await connectDB();
            const query = filter ? { fieldToFilter: new
               RegExp(filter, "i") } : {};
            const sort = sortField ? { [sortField]: sortOrder
               === "desc" ? -1 : 1 } : {};

            return await collection
              .find(query)
              .sort(sort)
              .skip(skip)
              .limit(take)
              .toArray();
        },

        item: async (_, { id }) => {
            const collection = await connectDB();
            return await collection.findOne({ _id: new
               ObjectId(id) });
        },
    },

    Mutation: {
        createItem: async (_, { fieldOne, fieldTwo }, { event
           }) => {

            const collection = await connectDB();
            const result = await collection.insertOne({
               fieldOne, fieldTwo });
            return { id: result.insertedId, fieldOne, fieldTwo };
        },

        updateItem: async (_, { id, fieldOne, fieldTwo }, {
           event }) => {
```

```
            const collection = await connectDB();
            const updateFields = {};

            if (fieldOne) updateFields.fieldOne = fieldOne;
            if (fieldTwo) updateFields.fieldTwo = fieldTwo;

            await collection.updateOne({ _id: new ObjectId(id)
               }, { $set: updateFields });
            return { id, ...updateFields };
        },

        deleteItem: async (_, { id }, { event }) => {

            const collection = await connectDB();
            const result = await collection.deleteOne({ _id: new
               ObjectId(id) });
            return result.deletedCount > 0;
        },
    },
};
```

4. Export the function:

```
const app = express();

const server = new ApolloServer({
    typeDefs,
    resolvers,
    context: ({ event }) => ({ event }), // pass request event
       to resolvers
});

// Start Apollo Server asynchronously
async function startApolloServer() {
    await server.start();
    server.applyMiddleware({ app });
}

startApolloServer(); // Initialize Apollo Server

// Export handler immediately after initializing the server
```

```
exports.handler = serverless(app);
```

5. Edit .toml file to introduce redirection:

```
[build]
functions = "functions"

[[redirects]]
  from = "/graphql"
  to = "/.netlify/functions/graphql"
  status = 200
```

6. Push these changes to your git repository.

7. Try querying and mutations against your deployment. Example query:

```
query {
  items(filter: "test", skip: 0, take: 5) {
    _id
    fieldOne
    fieldTwo
  }
}
```

8. Introduce query and mutation to your ui. You can use Apollo package if you use React, or just via plane POST requests of the format:

```
const url =
   'https://your-netlify-site.netlify.app/.netlify/functions/graphql';

const query = `
  query GetItems($filter: String) {
    items(filter: $filter) {
      _id
    fieldOne
    fieldTwo
    }
  }
`;

const variables = {
```

```
  filter: 'test', // Replace with the desired value or leave
     null for all values
};

fetch(url, {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
  },
  body: JSON.stringify({
    query,
    variables,
  }),
})
  .then(response => response.json())
  .then(data => {
    console.log('Response data:', data);
  })
  .catch(error => {
    console.error('Error fetching data:', error);
  });
```

Laboratory assignment 3: Authentication with JWT. Security.

JWT (JSON Web Token) is a compact and secure way of transmitting information between parties as a JSON object. It is commonly used for authentication and authorization in web applications. A JWT consists of three parts: header, payload, and signature, ensuring data integrity and verification. Since JWTs are stateless, they allow authentication without needing to store session data on the server.

Implementing JWT-based authentication in your netlify backend:

---

1. Install neccessary npm package:

   `npm install jsonwebtoken`

   The jsonwebtoken npm package is a popular library for working with JWTs in Node.js. It provides a set of methods for creating, signing, and verifying JWTs.

   After installing the package via console, verify that it has been added to `node_modules` folder, and in packages.json file.

2. Implement login.js, endpoint that would handle authorization of the user with a given username and password:

```
const jwt = require("jsonwebtoken");

exports.handler = async (event) => {
  if (event.httpMethod === "OPTIONS") {
    return {
      statusCode: 200,
      headers: {
        "Access-Control-Allow-Origin": "*", // change to your
          frontend domain
        "Access-Control-Allow-Methods": "GET, POST, PUT,
          DELETE, OPTIONS",
        "Access-Control-Allow-Headers": "Content-Type",
      },
      body: "",
    };
  }

  if (event.httpMethod !== "POST") {
    return { statusCode: 405, body: "Method Not Allowed" };
```

```
  }

  const { username, password } = JSON.parse(event.body);

  if (username !== process.env.USERNAME || password !==
     process.env.PASSWORD) {
    return { statusCode: 401, body: JSON.stringify({ error:
       "Invalid credentials" }) };
  }

  const secretKey = process.env.JWT_SECRET;
  const token = jwt.sign({ username }, secretKey, { expiresIn:
     "1h" });

  return {
    statusCode: 200,
        headers: {
          "Access-Control-Allow-Origin": "*", // change to your
             frontend domain
          "Access-Control-Allow-Methods": "GET, POST, PUT,
             DELETE, OPTIONS",
          "Access-Control-Allow-Headers": "Content-Type"
        },
    body: JSON.stringify({ token }),
  };
};
```

Do not also forget to set these username, password and key in .env file:

```
USERNAME=hello-user
PASSWORD=password1234
JWT_SECRET=secret-key
```

3. Implement authorization check on sensitive endpoints, for example, any updates to data:

```
const jwt = require("jsonwebtoken");

exports.handler = async (event) => {
  if (event.httpMethod === "OPTIONS") {
    return {
```

```
      statusCode: 200,
      headers: {
        "Access-Control-Allow-Origin": "*", // change to
           frontend domain in future
        "Access-Control-Allow-Methods": "GET, POST, PUT,
           DELETE, OPTIONS",
        "Access-Control-Allow-Headers": "Content-Type",
      },
      body: "",
    };
  }

  const token = event.headers.authorization?.split(" ")[1];
  const secretKey = process.env.JWT_SECRET;

  if (!token) {
    return { statusCode: 401, body: JSON.stringify({ error:
       "Unauthorized: No token provided" }) };
  }

  try {
    jwt.verify(token, secretKey);
  } catch (error) {
    return { statusCode: 403, body: JSON.stringify({ error:
       "Forbidden: Invalid token" }) };
  }

  // further operations for authorized user


}
```

4. Implement registration for new users, generation of tokens for users (not based on env variables, but on actual credentials, provided). Do not forget about password hashing.
5. Example on how to use login endpoint:

```
async function requestBackend(){
    let payload = {
        username: "username",
```

```
        password: "password"
    }

    let response = await
       fetch('https://your-domain/.netlify/functions/login', {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json'
        },
        body: JSON.stringify(payload)
    });

    if(!response.ok){
        console.error(response);
    }

    let data = await response.json();
    token = data.token;
    role = data.role;
    console.log(`role: ${role}`);

    if(role === "admin"){
        await addItem();
    }
}
```

---

6. Bonus task: find security vulnerabilities in other students projects. If found any, please mark them, and rise as an open issue in the students github repository. Accurately describe the issue, and possible fixes for it. Open a pull request with your own fix for the issue.

Laboratory assignment 4: WebSockets and WebHooks. Unit and Integration testing.

1.
   -
   -
2.
   -
   -
3.
   -
   -
4.
   -
   -
5.
   -
   -
6.
   -
   -
7.
   -
   -